# Cloudy Day Report

**Group 21**
*Evaluating Target Group 9*

June 4, 2026

## Diogo Sousa

### Documentation Feedback

- **Style Discrepancy:** The query parameter for pagination size is documented as `pageSize` (camelCase) while the response object uses `page_size` (snake_case). The API Gateway correctly accepts the `pageSize` parameter and returns the corresponding schema, but this inconsistency should ideally be unified.
- **Error Response Documentation:** While standard HTTP error codes (400, 401, 404, 500, 502) are consistently mapped across endpoints using reusable components (e.g., `BadRequestText`), they could be more descriptive. The schema indicates they return a `text/plain` string without explaining *what* triggers them (e.g., what specific validation failure causes a 400, or under what conditions a 404 is thrown for a specific endpoint).

### Testing Results

#### Smoke Testing (Endpoint: GET /api/listings/search)

The smoke test was executed using a Python script checking the endpoint. The test validated the HTTP 200 response, JSON validity, and the presence of all required top-level fields (`total`, `page`, `page_size`, `listings`) and listing level attributes.

- **Basic Paginated Search:** Passed. Returned 5 listings with a reported total of 50,000 records. Latency: $\approx$ 141ms.
- **Filtered Search (Toyota Corolla):** Passed. Returned 5 listings with a reported total of 300 records. Latency: $\approx$ 191ms.

No broken links or malfunctioning behaviors were detected.

#### Stress Testing (Endpoint: GET /api/listings)

The stress test was executed using Locust against endpoint.

- **Concurrency Settings:** 100 concurrent users, spawning at a rate of 20 users per second, run duration of 30 seconds.
- **Performance observed:**
  - **Total Requests:** 6,899
  - **Failures:** 0 (0.00%)

- **Requests Per Second (RPS):** Peaked at ≈ 275 req/s (average 248.81 req/s over the duration).
- **Latency Statistics:** Average = 72ms, Median = 62ms, 95th percentile = 93ms, 99th percentile = 150ms.

The endpoint proved to be highly stable under the tested load.

---

## Bruno Faustino

### Documentation Feedback

- **Some positives:**
  - The API documentation was in general easy to understand due to the modularity on the groups of endpoints and the descriptive information tagged with each endpoint;
  - The introductory paragraph gives a quick overview of all the use cases of the application and in which modules they can be found;
  - In general, the exceptions portray in which cases the app may fail (although they could be explained in more detail according to the endpoint in some cases).
- **A few suggestions:**
  - For the chat endpoints, a 404 code should be thrown if the chat id does not exist in the database;
  - It would be helpful to have default values already filled-in in the body of the POST methods that serve as an example to further improve the understandability of the API (for some fields it is not trivial what the expected format is, like fuel_type, drive_type and end_time);
  - Although it is well explained that the user needs to add the Host header before calling the post/put/delete endpoints, it is never explained that the user needs to be logged in and that the resulting token needs to be added to the header: a quick mention in either the introductory paragraph or the 401 Unauthorized exception in the documentation would generate less confusion and save a few 401s.

### Testing Results

#### Smoke Testing (Endpoint: GET /api/auctions/auction_id/bids)

The smoke test was executed using pytest and requests modules of Python.

Here is the sequence of API calls as a setup before testing the actual endpoint:

1. **POST /api/user/login** for the auth token
2. **POST /api/listings** for listing_id
3. **POST /api/auctions** for auction_id

4. 3x **POST /api/auctions/auction_id/bid**

After calling the endpoint: **GET /api/auctions/auction_id/bids**, it is expected that the command returns successfully (200) and with 3 auction bids in the created auction for the created listing.

**The smoke test passed!**

#### Stress Testing (Endpoint: GET /api/market/average_price)

The stress test was executed using Locust against the endpoint.

- **Concurrency Settings:** 10000 concurrent users, spawning at a rate of 5 users per second, running until failing.
- **Performance observed:**
  - **Total Requests:** 250
  - **Failures:** 4 (1.60%)
  - **Requests Per Second (RPS):** Very inconsistent, around 19.61 req/s.
  - **Latency Statistics:** Average = 1009ms, Median = 190ms, 95th percentile = 4000ms, 99th percentile = 6700ms.

The system fails a few times at 19.61 req/s due to the unavailability of an upstream service (502).

During other rounds of testing, the system could fail at less or more than that value, sometimes managing to survive without failures until around 50 req/s, so it is inconsistent, and the number of failures oscillates a lot between load tests, up to the point of reaching 99% failures.

The endpoint proved unable to sustain loads above 50 req/s, and shows signs of failing around 19.61 req/s.

---

## Rodrigo Neto

### Documentation Feedback

- **Missing Content-Type Header:** Successful JSON responses do not include the `Content-Type: application/json` header, which can break strict HTTP clients.
- **Poor Overload Behaviour:** Under high load, the backend does not gracefully degrade (e.g., return `429` or `503`). Instead, the gateway returns `502 Bad Gateway` after extremely long timeouts (up to 47 seconds).

### Testing Results

#### Smoke Testing (Endpoint: GET /api/market/price-comparison)

The smoke test was executed using Postman checking the endpoint. The test validated HTTP 200 responses, JSON validity, pagination, grouping, and sorting behaviors.

- **Happy Path:** Passed with warnings. Valid JSON, correct pagination and sorting. `Content-Type` assertion failed (header omitted).
- **Validation:** Partial pass (3/6). Missing parameters returned 400. Invalid enum values returned `502 Bad Gateway` instead of 400.

Some malfunctioning behaviors were detected during validation tests.

#### Stress Testing (Endpoint: GET /api/listings/stats/by-location)

The stress test was executed using Locust against the endpoint.

- **Concurrency Settings:** Step-load ramping from 10 to 200 concurrent users.
- **Performance observed:**
  - **Total Requests:** 3,524
  - **Failures:** 123 (3.49%)
  - **Requests Per Second (RPS):** Peaked at $\approx$ 50 req/s.
  - **Latency Statistics:** Average = 7,280ms, Median = 460ms, 95th percentile = 34,000ms, 99th percentile = 38,000ms.

The endpoint proved unable to sustain loads above $\approx$ 50 RPS, leaving the upstream backend in an unhealthy state.

---